IE437 Data-Driven Decision Making and Control
2024 Spring Semester

6. Data-Driven Design Optimization (Generative Model Based)

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Contents

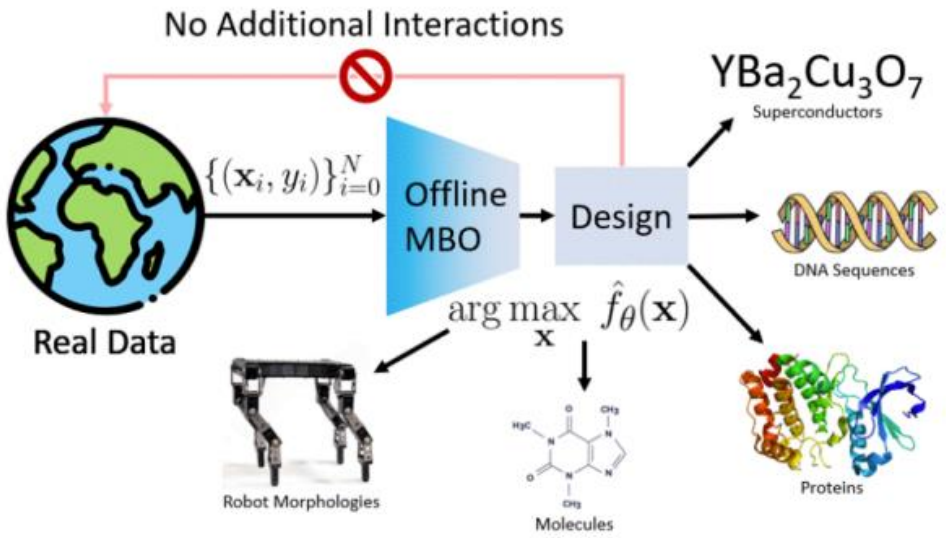
Papers fall into Inverse approaches

- Conditioning by Adaptive Sampling (CbAS, ICML 2019)
- Model Inversion Networks (MINs, NIPS 2020)

More Recent Papers

- Diffusion models for Black-box Optimization (DDOM, ICML 2023)
- Bootstrapped Training of Score-conditioned Generator for Offline Design of Biological Sequences (BootGen, NIPS 2023)

Motivation (Recap)



- In real world optimization problems, evaluation of objective function is expensive and prohibitive
  Examples) Molecule Design, Automated Aircraft Design, …

- Therefore, constructing a data-driven model given a fixed, offline dataset is crucial

- Formally, we try to find an input $\boldsymbol{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, f(\mathbf{x}) \text{ with } \textbf{only} \text{ a fixed dataset, } D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \ldots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$$

Motivation

- Formally, we try to find an input ***x*** such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, f(\mathbf{x}) \text{ with } \textbf{only} \text{ a fixed dataset, } D = \left\{\left(\mathbf{x}_1, f(\mathbf{x}_1)\right), \ldots, \left(\mathbf{x}_N, f(\mathbf{x}_N)\right)\right\}$$

- Main Approaches

1. Surrogate model-based: Approximate $f(x)$ using previously collected data, choose the query that maximizes the surrogate model

2. Generative model-based: Learn an inverse function $f^{-1}(y)$ using previously collected data, choose the query generated from the inverse function given by a desired output.

- Today, we'll focus on Generative model-based approaches

Motivation

- **Main Problem of Offline Model-Based Optimization:**
  - Input space is high-dimensional, but usually with narrow manifold for valid inputs (Protein Design, Natural Image, …)
  - Since training distribution shows a small fraction of input space, optimizer can easily fall into adversarial inputs which results in arbitrary outputs
- **Surrogate model based approach resolve these issues by "Conservatism"**
  - Limit the expressivity of the model and often fails to find a better design
- **Generative model based approach samples the distribution conditioned on the desired properties**

$$\text{Jointly model } p(\mathbf{x}, y) = p(y|\mathbf{x})p(\mathbf{x})$$

$$\text{Sample from } p(\mathbf{x}|y \geq y_{max}), \text{more generally}, p(\mathbf{x}|S)$$

  - Avoid solving optimization problem (i.e., gradient decent) but just sample (convenient)
  - Can provide a diversified designs (**high expressivity of generative models**, i.e., multi-modality)
  - Implicitly satisfy the constraints while sampling (trained only by the feasible designs)

Data-Driven Design Optimization (Generative Model-Based)

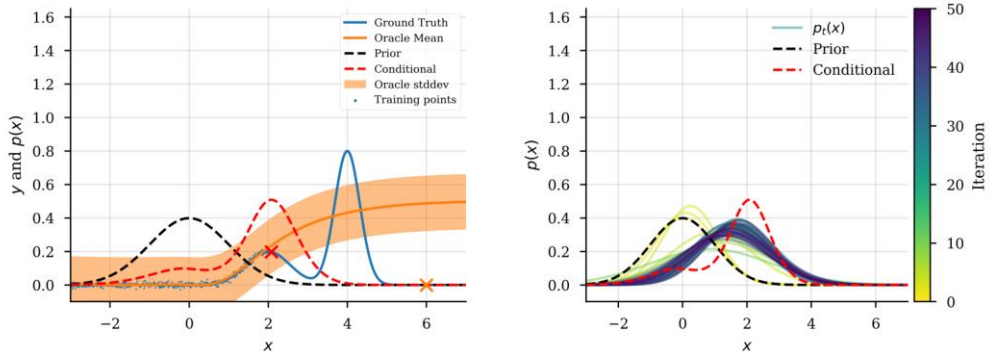
1. CbAS
2. MINs

1. Conditioning by adaptive sampling for robust design (CbAS)

Conditioning by adaptive sampling for robust design

David H. Brookes [1] **Hahnbeom Park** [2 3] **Jennifer Listgarten** [4]

Proceedings of the 36th International Conference on Machine Learning, Long Beach, California, PMLR 97, 2019. Copyright 2019 by the author(s).



1. Motivation

- **Main Problem of Offline Model-Based Optimization:**
  - Input space is high-dimensional, but usually with narrow manifold for valid inputs (Protein Design, Natural Image, …)
  - Since training distribution shows a small fraction of input space, optimizer can easily fall into adversarial inputs which results in arbitrary outputs

- **New Approach:**
  - Instead of learning predictive models, estimate distribution over the design space
  - Generate samples from the distribution conditioned on the desired properties

2. Target Problem

- **Problem Statement:**
  - Our objective is to find an input $\boldsymbol{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, f(\mathbf{x})$$

$$\text{with } \textbf{only} \text{ a fixed dataset, } D = \left\{\left(\mathbf{x}_1, f(\mathbf{x}_1)\right), \ldots, \left(\mathbf{x}_N, f(\mathbf{x}_N)\right)\right\}$$

  - Our Approach:

$$\text{Jointly model } p(\mathbf{x}, y) = p(y|\mathbf{x})p(\mathbf{x})$$

$$\text{Sample from } p(\mathbf{x}|y \geq y_{max}), \text{more generally, } p(\mathbf{x}|S)$$

- **Assumptions:**
  - have access to a property oracle, $p(y|\mathbf{x})$, which may be a black box function.
  - prior knowledge has been encoded in $p(\mathbf{x})$.
    - ✓ The prior density, $p(\mathbf{x})$, can be modelled by training an appropriate generative model, such as a Variational Auto-Encoder (VAE), GAN, Diffusion Model, etc.

3. Conditioning by Adaptive Sampling (CbAS)

- Our ultimate aim is to condition this prior on our desired property values, S, and sample from the resulting distribution:

$$p\left(\mathbf{x}|S,\boldsymbol{\theta}^{(0)}\right)=\frac{P(S|\mathbf{x})p\left(\mathbf{x}|\boldsymbol{\theta}^{(0)}\right)}{P\left(S|\boldsymbol{\theta}^{(0)}\right)}$$

  - ✓ Generating realizations of $\mathbf{x}$ that are likely to have low error in the predictive model or are realistic (i. e., are drawn from the underlying data distribution $p\left(\mathbf{x}|\boldsymbol{\theta}^{(0)}\right)$: modeled by expressive generative model
  - ✓ have high probability $P(S|\mathbf{x})$ of satisfying our desideratum encoded in S.

- Toward this end, we will assume that the desired conditional can be well-approximated with a sufficiently rich generative model, $q(\mathbf{x}|\phi)$, which need not be of the same parametric form as the prior.

$$\begin{aligned}\phi^{*}&=\underset{\phi}{\operatorname{argmin}}\,D_{KL}\left(p\left(\mathbf{x}|S,\theta^{(0)}\right)\middle\|q(\mathbf{x}|\phi)\right)\\&=\underset{\phi}{\operatorname{argmax}}\,\mathbb{E}_{p\left(\mathbf{x}|\theta^{(0)}\right)}[P(S|\mathbf{x})\log q(\mathbf{x}|\phi)]\end{aligned}$$

- As our algorithm iterates, the parameter, ϕ, will slowly move toward a value that best approximates our desired conditional density.
  - ✓ a similar flavor to that of variational inference

3. Conditioning by Adaptive Sampling (CbAS)

- Optimization Formulation in details:

$$\boldsymbol{\phi}^* = \underset{\boldsymbol{\phi}}{\operatorname{argmin}}\, D_{KL}\left(p(\mathbf{x}|S, \boldsymbol{\theta}^{(0)})||q(\mathbf{x}|\boldsymbol{\phi})\right) \tag{2}$$

$$= \underset{\boldsymbol{\phi}}{\operatorname{argmin}}\, -\mathbb{E}_{p(\mathbf{x}|S,\boldsymbol{\theta}^{(0)})}[\log q(\mathbf{x}|\boldsymbol{\phi})] - H_0 \tag{3}$$

$$= \underset{\boldsymbol{\phi}}{\operatorname{argmax}}\, \frac{1}{P(S|\boldsymbol{\theta}^{(0)})}\mathbb{E}_{p(\mathbf{x}|\boldsymbol{\theta}^{(0)})}[P(S|\mathbf{x}) \log q(\mathbf{x}|\boldsymbol{\phi})] \tag{4}$$

$$= \underset{\boldsymbol{\phi}}{\operatorname{argmax}}\, \mathbb{E}_{p(\mathbf{x}|\boldsymbol{\theta}^{(0)})}[P(S|\mathbf{x}) \log q(\mathbf{x}|\boldsymbol{\phi})], \tag{5}$$

where $H_0 \equiv -\mathbb{E}_{p(\mathbf{x}|S,\boldsymbol{\theta}^{(0)})}[\log p(\mathbf{x}|S, \boldsymbol{\theta}^{(0)})]$

3. Conditioning by Adaptive Sampling (CbAS)

- **Estimate Conditional Distribution**
  - Naïve approach: Draw samples $\mathbf{x}_i \sim p\left(\mathbf{x}|\theta^{(0)}\right)$ from $p\left(\mathbf{x}|\theta^{(0)}\right)$ to obtain MC approximation to the expectation
    - ✓ **Problem**: In Design optimization problem, satisfying S will be exceedingly rare, consequently, $P(S|\mathbf{x})$ will be vanishingly small for most $\mathbf{x}_i$ sampled from the $p\left(\mathbf{x}|\theta^{(0)}\right)$
    - ✓ an MC approximation will exhibit high variance and require an arbitrarily large number of samples to calculate accurately.
  - Alternative Approach: Importance Sampling

$$\mathbb{E}_{p\left(\mathbf{x}|\theta^{(0)}\right)}[P(S|\mathbf{x}) \log q(\mathbf{x}|\phi)]$$

$$= \mathbb{E}_{r(x)}\left[\frac{p\left(\mathbf{x}|\theta^{(0)}\right)}{r(x)} P(S|\mathbf{x}) \log q(\mathbf{x}|\phi)\right]$$

    - ✓ **Question**: How to find a good proposal distribution? Set $r^{(t)} = q\left(\mathbf{x}|\phi^{(t-1)}\right)$

3. Conditioning by Adaptive Sampling (CbAS)

$$\mathbb{E}_{p(\mathbf{x}|\theta^{(0)})}[P(S|\mathbf{x}) \log q(\mathbf{x}|\phi)] = \mathbb{E}_{r(x)}\left[\frac{p(\mathbf{x}|\theta^{(0)})}{r(x)} P(S|\mathbf{x}) \log q(\mathbf{x}|\phi)\right]$$

- Construct a series relaxed conditions, $S^{(t)}$, and corresponding importance sampling distributions, $r^{(t)}(x)$ such that:

A. $\mathbb{E}_{r^{(t)}(x)}\left[P\left(S^{(t)}|\mathbf{x}\right)\right]$ is non-vanishing

✓ can draw samples, $\mathbf{x}$, from $r^{(t)}(x)$ that have reasonably high values of $P(S|\mathbf{x})$.

A. $S^{(t)} \supset S^{(t+1)} \supset S$, for all t.

✓ slowly move toward our desired property condition; that is, it ensures that $S^{(t)}$ approaches S as t grows large

3. Conditioning by Adaptive Sampling (CbAS)

Initialization

A. $\mathbb{E}_{r^{(t)}(x)}\left[P\left(S^{(t)}|\mathbf{x}\right)\right]$ is non-vanishing

✓ can draw samples, $\mathbf{x}$, from $r^{(t)}(x)$ that have reasonably high values of $P(S|\mathbf{x})$.

To achieve this,

(1) set $S^{(t)}$ to be the relaxed condition that $p(y \geq \gamma^{(0)}|\mathbf{x})$ where $\gamma^{(0)}$ is the Qth percentile of property values predicted for those samples used to construct the prior

(2) set the proposal distribution to the prior, $r^{(t)}(x) = p(\mathbf{x}|\boldsymbol{\theta}^0)$.

This is how we set the first tuple of the required sequence, ($S^{(t)}$, $r^{(t)}$).

Iterative update on $q\left(\mathbf{x}|\phi^{(t)}\right)$ with incrementally adjusted target $S^{(0)}$,..., $S^{(t)}$

$$q\left(\mathbf{x}|\phi^{(0)}\right) = \underset{q(\mathbf{x}|\phi)}{\operatorname{argmax}}\, \mathbb{E}_{p\left(\mathbf{x}|\boldsymbol{\theta}^0\right)}\left[\frac{p\left(\mathbf{x}|\theta^{(0)}\right)}{p\left(\mathbf{x}|\boldsymbol{\theta}^0\right)} P\left(S^{(0)}|\mathbf{x}\right) \log q(\mathbf{x}|\phi)\right]$$

$$q\left(\mathbf{x}|\phi^{(1)}\right) = \underset{q(\mathbf{x}|\phi)}{\operatorname{argmax}}\, \mathbb{E}_{p\left(\mathbf{x}|\boldsymbol{\theta}^0\right)}\left[\frac{p\left(\mathbf{x}|\theta^{(0)}\right)}{q\left(\mathbf{x}|\phi^{(0)}\right)} P\left(S^{(1)}|\mathbf{x}\right) \log q(\mathbf{x}|\phi)\right]$$

$S^{(t)}$ to be the relaxed condition that $p(y \geq \gamma^{(0)}|\mathbf{x})$ from the samples from the previous iteration

3. (CbAS)

$$\mathbb{E}_{p(\mathbf{x}|\theta^{(0)})}[P(S|\mathbf{x})\log q(\mathbf{x}|\phi)] = \mathbb{E}_{r(x)}\left[\frac{p(\mathbf{x}|\theta^{(0)})}{r(x)}P(S|\mathbf{x})\log q(\mathbf{x}|\phi)\right] \quad \text{(Importance Sampling)}$$

$$= \mathbb{E}_{q(\mathbf{x}|\phi^{(t)})}\left[\frac{p(\mathbf{x}|\theta^{(0)})}{q(\mathbf{x}|\phi^{(t)})}P(S|\mathbf{x})\log q(\mathbf{x}|\phi)\right] \quad \text{(Set Proposal Distribution)}$$

$$\approx \sum_{i=1}^{M}\frac{p\left(\mathbf{x}_i^{(t)}|\theta^{(0)}\right)}{q\left(\mathbf{x}_i^{(t)}|\phi^{(t)}\right)}P\left(S|\mathbf{x}_{\boldsymbol{i}}^{(t)}\right)\log q\left(\mathbf{x}_i^{(t)}|\phi\right) \quad \text{(MC Approximation)}$$

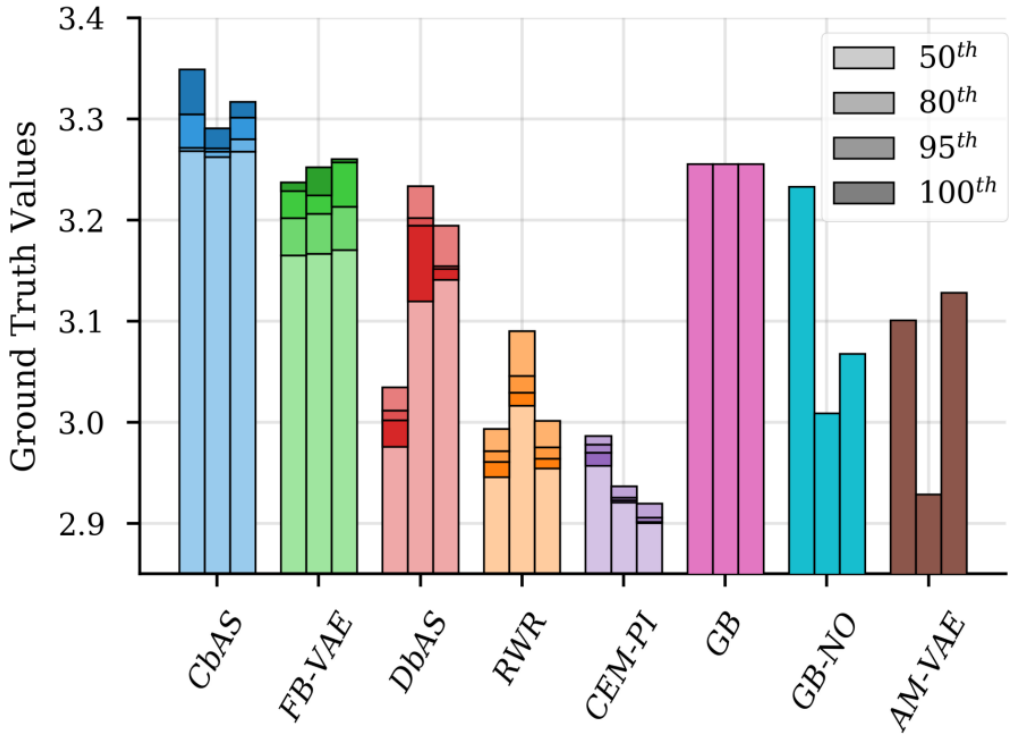
$$\phi^{(t+1)} = \underset{\phi}{\operatorname{argmax}}\sum_{i=1}^{M}\frac{p\left(\mathbf{x}_i^{(\boldsymbol{t})}|\theta^{(0)}\right)}{q\left(\mathbf{x}_i^{(\boldsymbol{t})}|\phi^{(t)}\right)}P\left(S^{(t)}|\mathbf{x}_{\boldsymbol{i}}^{(\boldsymbol{t})}\right)\log q\left(\mathbf{x}_i^{(t)}|\phi\right)$$

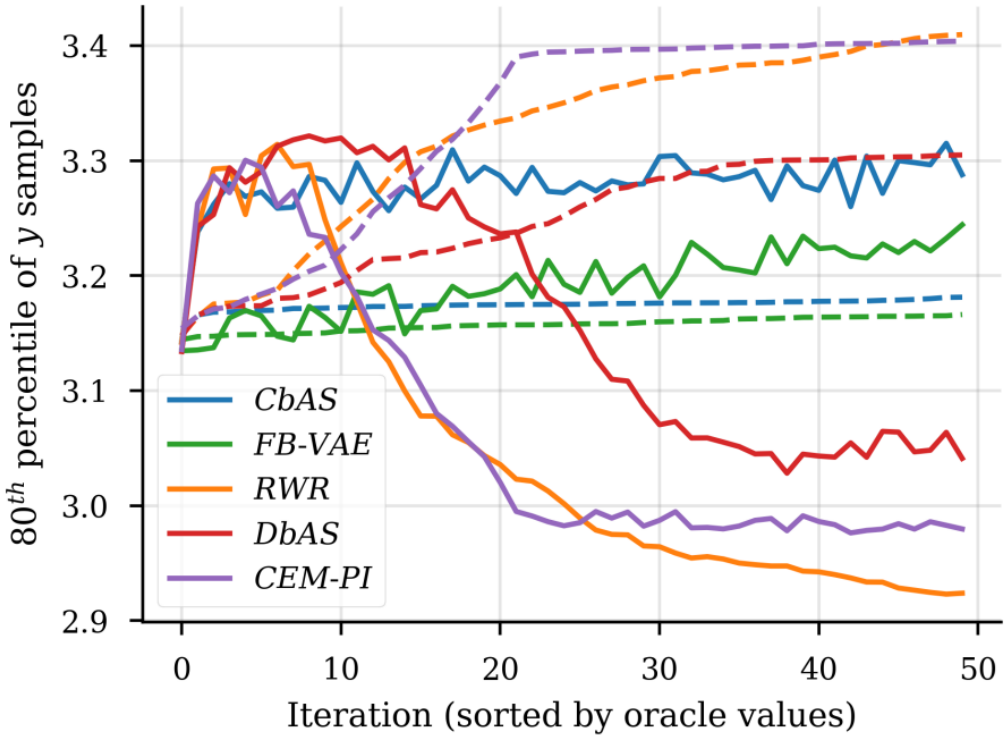
- A low-variance estimate of the target objective, which can be viewed as a weighted maximum likelihood with weights given by $\frac{p\left(\mathbf{x}_i^{(\boldsymbol{t})}|\theta^{(0)}\right)}{q\left(\mathbf{x}_i^{(\boldsymbol{t})}|\phi^{(t)}\right)}P\left(S^{(t)}|\mathbf{x}_{\boldsymbol{i}}^{(\boldsymbol{t})}\right)$.
- The objective function can now be optimized using any number of standard techniques for training generative models.

4. Experiments

- **Protein Fluorescence**

  Goal: Maximizing protein fluorescence
  Input Dimension: Combinatorial space (20^237)
  Offline Dataset: 5,000 samples from the original data



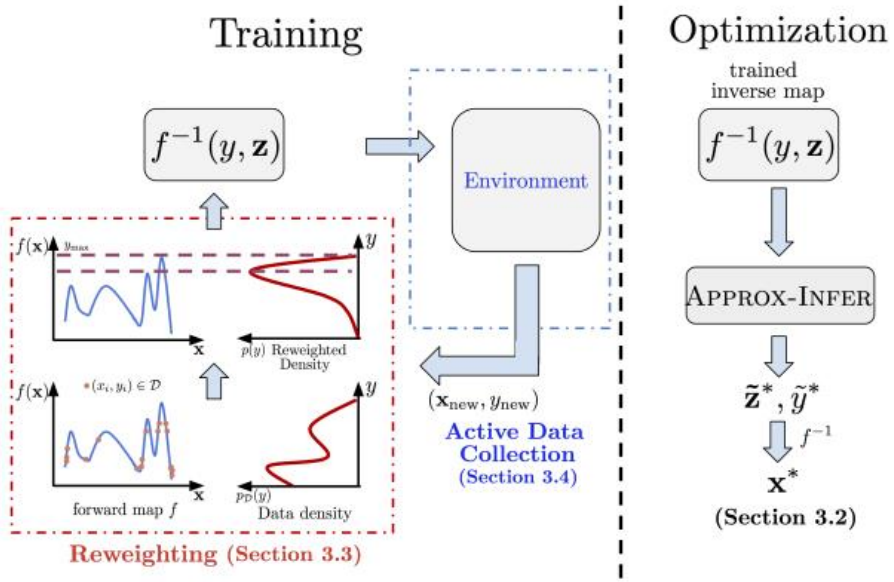


2. Model Inversion Networks for Model-based Optimization (MINs)

Model Inversion Networks for Model-Based Optimization

Aviral Kumar, Sergey Levine
Electrical Engineering and Computer Sciences, UC Berkeley
aviralk@berkeley.edu

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.



1. Motivation

- **Main Problem of Offline Model-Based Optimization:**
  - Input space is high-dimensional, but usually with narrow manifold for valid inputs (Protein Design, Natural Image, …)
  - Since training distribution shows a small fraction of input space, optimizer can easily fall into adversarial inputs which results in arbitrary outputs

- **New Approach:**
  - Since the high-dimensionality is problematic, how about learning an inverse mapping?
  - Since the input value is scalar, inverse mapping does not suffer from high-dimensionality
  - Furthermore, we can exploit various techniques from deep generative models to model high-dimensional data distribution

$$f_\theta : \mathcal{X} \to \mathcal{Y} \longrightarrow f_\theta^{-1} : \mathcal{Y} \to \mathcal{X}$$

2. Target Problem

- **Problem Statement:**
  - Our objective is to find an input $\boldsymbol{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, f(\mathbf{x})$$

$$\text{with } \textbf{only} \text{ a fixed dataset, } D = \left\{\left(\mathbf{x}_1, f(\mathbf{x}_1)\right), \ldots, \left(\mathbf{x}_N, f(\mathbf{x}_N)\right)\right\}$$

3. Model Inversion Networks(MINs)

- **Optimization via Inverse Maps**
  - We want to learn an inverse mapping

Question 1) Multiple x can correspond to the same y

Answer 1) Make an inverse mapping stochastic by introducing random variable z (cf. CVAE)

$$f_\theta^{-1} : \mathcal{Y} \to \mathcal{X}$$

$$f_\theta^{-1} : \mathcal{Y} \times \mathcal{Z} \to \mathcal{X}, \text{ where } \mathbf{z} \sim p_0(\mathbf{z}), \text{ a prior distribution}$$

3. Model Inversion Networks(MINs)

- **Optimization via Inverse Maps**
  - Now we can train the inverse map by minimizing the following objective:

$$L_p(D) = \mathrm{E}_{y\sim p(y)}\left[Div\left(p_D(\mathbf{x}|y), p_{f_\theta^{-1}}(\mathbf{x}|y)\right)\right]$$

Div is a measure of divergence between two distributions.

Option 1) KL Divergence → Maximum Log Likelihood

Option 2) JS Divergence → GAN-Style Training Objective

3. Model Inversion Networks(MINs)

- **Reweighting the Training Distribution**
  - Recall our training objective:

$$L_p(D) = \mathrm{E}_{y \sim p(y)} \left[Div \left(p_D(\mathbf{x}|y), p_{f_\theta^{-1}}(\mathbf{x}|y) \right) \right]$$

We sample ***y*** from the static data distribution

Option 1) Uniform Sampling
→ Straightforward, but our goal is optimization. Accurate predictions for high values are much important

Option 2) Greedy Sampling
→ Training with the highest value is appealing, but leads to high variance

Option 3) Importance Sampling
→ To trade-off bias and variance, we re-weight the training points with importance weight

$$\hat{\mathcal{L}}_p(\mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_i \mathbf{w}_i \cdot \hat{D}(\mathbf{x}_i, f_\theta^{-1}(y_i))$$

3. Model Inversion Networks(MINs)

- **Inference with Inverse Maps (Approx-Infer)**
  - Now we want to choose an input that will maximize the black-box function
  - Since we utilize the inverse map, we should carefully choose ***y*** to query.

Option 1) choose the best ***y*** from the static dataset
→ Reasonable, but we should be able to extrapolate beyond the best score in the static dataset

Option 2) using independently trained forward model
→ by **restricting generated input from the inverse map is predicted to have similar score to y**, we can extrapolate as far as possible while staying on the data manifold

Formally, we choose ***y, z*** to query with(z is also constrained to have high probability under the prior):

$$\tilde{y}^*, \tilde{\mathbf{z}}^* = \underset{y,\mathbf{z}}{\operatorname{argmax}} f_\theta\left(f_\theta^{-1}(y,\mathbf{z})\right) \; s.t \; \left\|y - f_\theta\left(f_\theta^{-1}(y,\mathbf{z})\right)\right\|^2 \leq \epsilon_1, \quad p(\mathbf{z}) \geq \epsilon_2$$

Find maximum

Prevent going not to far

3. Model Inversion Networks(MINs)

- **Reweighting the Training Distribution**
  - Calculate importance weights

Theorem 3.1 (Bias + variance bound in MINs). *Let $\mathcal{L}(p^*)$ be the expected objective value under the Dirac-delta distribution, $p^*(y) = \mathbf{1}[y = y^*]$, centered at the optimal $y^* \in \mathcal{D}$: $\mathcal{L}(p^*) = \mathbb{E}_{y\sim p^*(y)}[D(p(\mathbf{x}|y), f^{-1}(y))]$. Let N_y be the number of datapoints with the particular y value observed in $\mathcal{D}$, and d_2 denote the exponentiated Renyi divergence For some constants C_1, C_2, C_3, with high confidence,*

$$\mathbb{E}\left[||\nabla_\theta \hat{\mathcal{L}}_p(\mathcal{D}) - \nabla_\theta \mathcal{L}(p^*)||_2^2\right] \leq C_1 \mathbb{E}_{y\sim p(y)}\left[\frac{1}{N_y}\right] + C_2 \frac{d_2(p||p_\mathcal{D})}{|\mathcal{D}|} + C_3 \cdot D_{\mathrm{TV}}(p^*, p)^2.$$

According to Theorem 3.1, importance weight is assigned as

$$\mathbf{w}_i = \frac{p(y)}{p_D(y)}, where\ p(y) \propto \frac{N_y}{N_y + K} \cdot exp(y - y^*)$$

$$N_y = \text{number of datapoints with score } y$$
$$y^\star = \text{highest score in dataset}$$

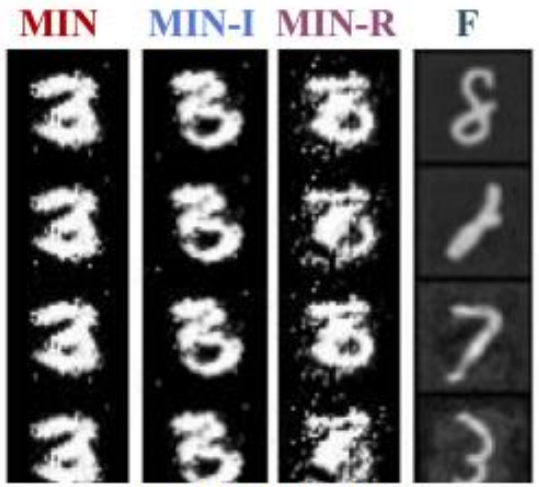
Intuitively, we **upweights points with high y-values, provided the number of samples at that score, is not too low**

4. Experiments

- **Character Stroke Width Optimization**

Goal: Produce images with thickest stroke width that is still recognizable
(define stroke width as the number of pixels with intensity more than a threshold (= 0.2) in the image)

Input Dimension: 32×32 = 1024 image pixels
Offline Dataset: MNIST dataset



(b) Thickest 3

Task	Dimensions	Dataset (avg)	Dataset (best)	Forward map	MIN (Ours)
MINST (a)	1024	149.0	265.0	Invalid	**276.3**
MNIST (b)	1024	149.0	163.0	Invalid	**234.3**
MNIST (c)	1024	1.6	**3.0**	1.4	2.3

4. Experiments

- **Semantic Image Optimization**

  Goal: Produce images with youngest faces
  (evaluated by subjective judgement from 13 human subjects)

  Input Dimension: 64×64×3 = 12288 dimension space
  Offline Dataset 1) IMDB-Wiki faces dataset with all faces older than 15
  Offline Dataset 2) Same dataset with all faces older than 25

Task	Dimensions	Dataset (avg)	Dataset (best)	Forward map	MIN (Ours)
Faces ($\geq$ 15)	12288	38.7	-15.0	Invalid	**-12.2**
Faces ($\geq$ 25)	12288	41.5	-25.0	Invalid	**-23.9**

4. Experiments

- **Neural Network Parameter Optimization**

  Goal: Maximize the total reward obtained when the parametrized controller is executed in the simulator

  Input Dimension: 3843(Hopper), 1537(Pendulum) dimension
  Offline Dataset: 552(Hopper), 3480(Pendulum) set of pairs

Task	Dimensions	Dataset (avg)	Dataset (best)	Forward map	MIN (Ours)
Hopper	3843	442.9	1915.5	93.1	**1960.1**
Pendulum	1537	14.7	344.5	3.4	**1000.0**

4. Experiments

- **Ablation Studies**

Conduct ablation studies for identifying the benefits of different components of MINs.

1) MINs w/o Approx-Infer – Use the best score y to choose an input x
2) MINs w/o Re-weighting – Uniform Sampling when training

Approx-Infer and Re-weighting improves the accuracy, indicating that both components are essential

Dataset & Type	BanditNet	BanditNet*	MIN w/o I	MIN (Ours)	MINs w/o R
MNIST (49% corr.)	36.42 ± 0.6	–	94.2 ± 0.13	$\mathbf{95.0 \pm 0.16}$	95.0 ± 0.21
MNIST (Uniform)	9.94 ± 0.0	–	92.21 ± 0.22	$\mathbf{93.67 \pm 0.51}$	92.8 ± 0.01
CIFAR-10 (49% corr.)	42.13 ± 2.35	87.0	91.35 ± 0.87	$\mathbf{92.21 \pm 1.0}$	89.02 ± 0.05
CIFAR-10 (Uniform)	14.43 ± 1.43	–	76.31 ± 0.40	$\mathbf{77.12 \pm 0.54}$	74.87 ± 0.12

Data-Driven Design Optimization (Generative Model-Based)

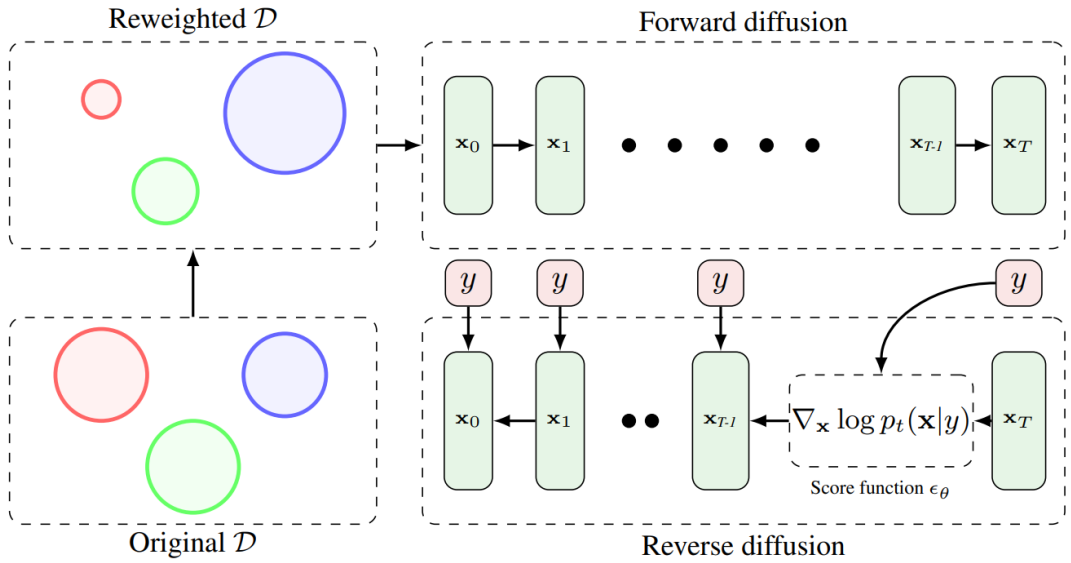
Cutting-Edge Papers (After 2022)

1. Diffusion Models for Black-box Optimization (DDOM)

Diffusion Models for Black-Box Optimization

Siddarth Krishnamoorthy [1] **Satvik Mashkaria** [1] **Aditya Grover** [1]

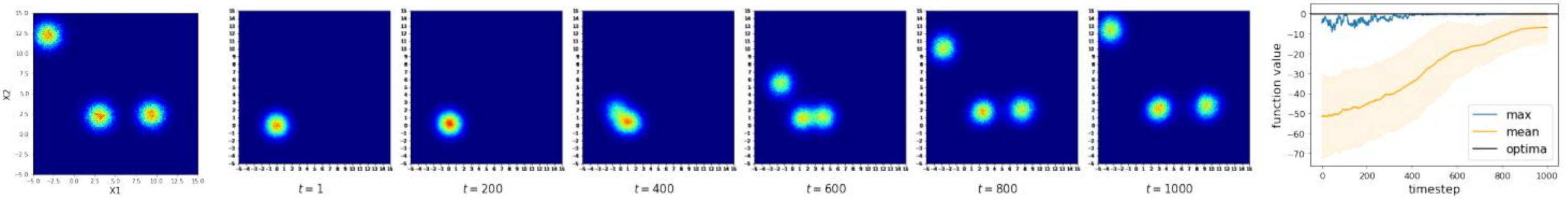
Proceedings of the 40^{th} International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 202, 2023. Copyright 2023 by the author(s).



1. Motivation

- **Main Problem of Offline Model-Based Optimization with Generative model-based approaches:**
  - Inverse mapping is inherently one-to-many mapping (same y but different x values)
  - It is difficult to learn such mapping in high-dimensional setting

- **New Approach:**
  - How about learning an inverse mapping with diffusion?
  - Diffusion model can cover multi-model distribution



(a) $\mathcal{D}_{\text{GMM}}$ (b) Sample reverse diffusion trajectory for DDOM trained on $\mathcal{D}_{\text{GMM}}$ (c) Sample statistics

2. DDOM

- **Step 1: Data Reweighting**
  - Similar to MINs, DDOM utilized reweighting to focus on high-scoring regions

$$\mathbf{w}_i = \frac{p(y)}{p_D(y)}, where\ p(y) \propto \frac{N_y}{N_y + K} \cdot exp(y - y^*)$$

N_y = number of datapoints with score y

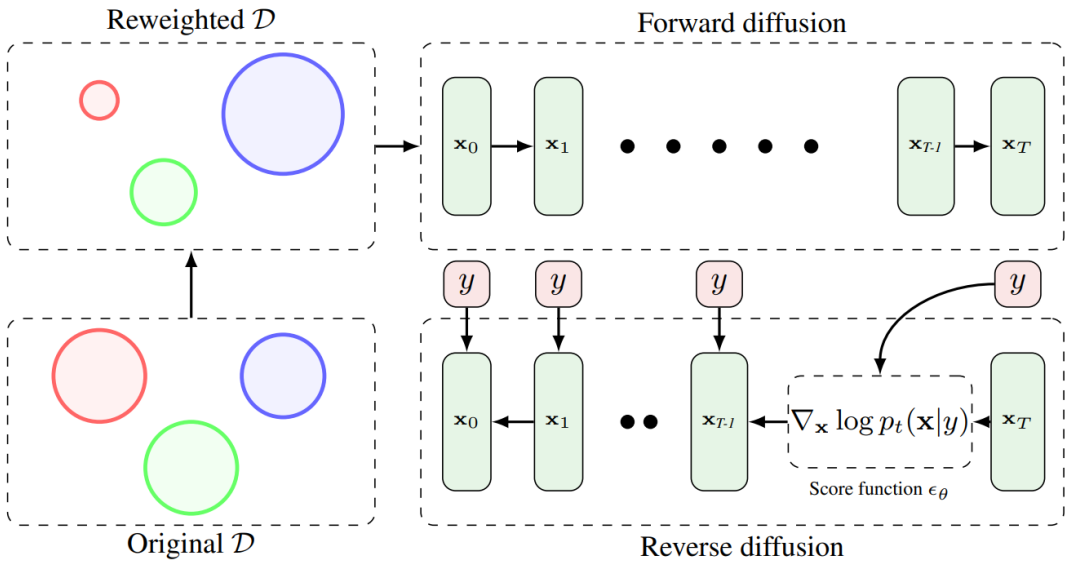
$y^\star$ = highest score in dataset

	D'KITTY	ANT	TFBIND8	TFBIND10	SUPERCON.	CHEMBL
No reweighting	0.926 ± 0.008	0.888 ± 0.018	0.957 ± 0.000	0.644 ± 0.011	0.553 ± 0.051	$\mathbf{0.633 \pm 0.000}$
Reweighting	$\mathbf{0.930 \pm 0.003}$	$\mathbf{0.960 \pm 0.015}$	$\mathbf{0.971 \pm 0.005}$	$\mathbf{0.688 \pm 0.092}$	$\mathbf{0.560 \pm 0.044}$	$\mathbf{0.633 \pm 0.000}$

Table 2. Comparison of normalized scores for DDOM with and without reweighting on Design-Bench. Higher is better We find that there is a significant improvement in score from reweighting on most tasks. We report scores averaged across 5 seeds.

2. DDOM

- **Step 2: Train Conditional Diffusion Model**
  - Train conditional diffusion model



- **Step 3: Sampling via Classifier-free Guidance**
  - Utilizing classifier-free guidance to sample high-scoring input points.

$$\epsilon_\theta(\mathbf{x}, t, y) = (1+\gamma)\epsilon_{\text{cond}}(\mathbf{x}, t, y) - \gamma\epsilon_{\text{uncond}}(\mathbf{x}, t)$$

3. Experiment Results

- **Outperform state-of-the-art baselines**

BASELINE	TFBIND8	TFBIND10	SUPERCON.	ANT	D'KITTY	CHEMBL	MEAN SCORE	MEAN RANK
$\mathcal{D}$ (best)	0.439	0.467	0.399	0.565	0.884	0.605	-	-
CbAS	0.958 ± 0.018	0.657 ± 0.017	0.45 ± 0.083	0.876 ± 0.015	0.896 ± 0.016	0.640 ± 0.005	0.746 ± 0.003	5.5
GP-qEI	0.824 ± 0.086	0.635 ± 0.011	0.501 ± 0.021	0.887 ± 0.0	0.896 ± 0.0	0.633 ± 0.000	0.729 ± 0.019	6.2
CMA-ES	0.933 ± 0.035	**0.679 ± 0.034**	0.491 ± 0.004	**1.436 ± 0.928**	0.725 ± 0.002	0.636 ± 0.004	**0.816 ± 0.168**	4.5
Gradient Ascent	**0.981 ± 0.015**	0.659 ± 0.039	**0.504 ± 0.005**	0.34 ± 0.034	0.906 ± 0.017	0.647 ± 0.020	0.672 ± 0.021	**3.5**
REINFORCE	0.959 ± 0.013	0.64 ± 0.028	0.481 ± 0.017	0.261 ± 0.042	0.474 ± 0.202	0.636 ± 0.023	0.575 ± 0.054	6.3
MINs	0.938 ± 0.047	0.659 ± 0.044	0.484 ± 0.017	0.942 ± 0.018	**0.944 ± 0.009**	**0.653 ± 0.002**	0.770 ± 0.023	**3.5**
COMs	0.964 ± 0.02	0.654 ± 0.02	0.423 ± 0.033	0.949 ± 0.021	**0.948 ± 0.006**	**0.648 ± 0.005**	0.764 ± 0.018	3.7
DDOM	**0.971 ± 0.005**	**0.688 ± 0.092**	**0.560 ± 0.044**	**0.957 ± 0.012**	0.926 ± 0.009	0.633 ± 0.007	**0.787 ± 0.034**	**2.8**

Table 1. Comparative evaluation of DDOM over 6 tasks, with each task averaged over 5 seeds. We report normalized results (along with stddev) with a budget $Q = 256$. We highlight the top two results in each column (and where there is a tie, we highlight both). **Blue** refers to the best entry, and **Violet** refers to the second best. We find that DDOM achieves the best average rank of all the baselines and is second best on the mean score metric.

2. Bootstrapped Training of Score-conditioned Generator for Offline Design of Biological Sequences (BootGen)

Bootstrapped Training of Score-Conditioned Generator for Offline Design of Biological Sequences

Minsu Kim[1] **Federico Berto**[1] **Sungsoo Ahn**[2] **Jinkyoo Park**[1]
[1]Korea Advanced Institute of Science and Technology (KAIST)
[2]Pohang University of Science and Technology (POSTECH)
{min-su, fberto, jinkyoo.park}@kaist.ac.kr
{sungsoo.ahn}@postech.ac.kr

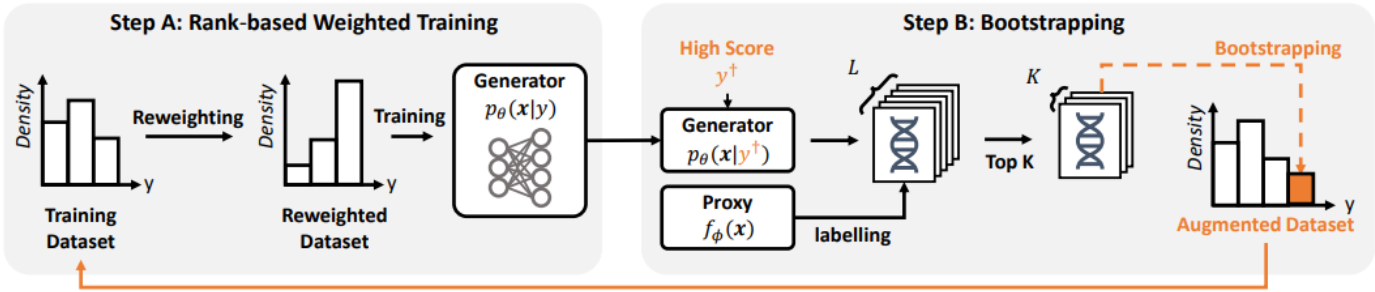


Figure 2.1: Illustration of the bootstrapped training process for learning score-conditioned generator.

1. Motivation

- **Main Problem of Offline Model-Based Optimization with Generative model-based approaches:**
  - While generative modeling is an appealing framework, it sometimes gives us bad results (lack of generalizability on high-scoring regions)
  - How can we develop better generative model-based approaches for offline model-based optimization?

- **New Approach:**
  - Step 1: Rank-based Reweighting
  - Step 2: Bootstrapping

2. BootGen

- **Step 1: Rank-based Reweighting**
  - Prior Works (MINs, DDOM): utilize score-based reweighting

$$\mathbf{w}_i = \frac{p(y)}{p_D(y)}, where\ p(y) \propto \frac{N_y}{N_y + K} \cdot exp(y - y^*)$$

N_y = number of datapoints with score y

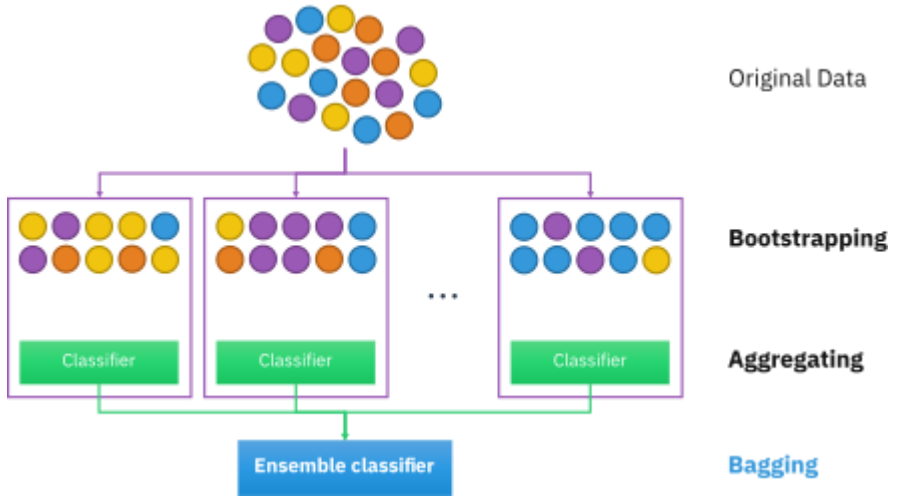
$y^\star$ = highest score in dataset

  - BootGen: Utilize rank-based reweighting

$$\mathcal{L}(\theta) := -\sum_{(\boldsymbol{x},y)\in\mathcal{D}_{\text{tr}}} w(y,\mathcal{D}_{\text{tr}}) \log p_\theta(\boldsymbol{x}|y), \quad w(y,\mathcal{D}_{\text{tr}}) = \frac{(k|\mathcal{D}_{\text{tr}}| + \text{rank}(y,\mathcal{D}_{\text{tr}}))^{-1}}{\sum_{(\boldsymbol{x},y)\in\mathcal{D}_{\text{tr}}}(k|\mathcal{D}_{\text{tr}}| + \text{rank}(y,\mathcal{D}_{\text{tr}}))^{-1}}.$$

2. BootGen

- **Step 2: Bootstrapping**
  - We want to focus on high-scoring regions
  - However, exploiting a single trained model may be dangerous (Fall into OOD)
  - In this paper, we utilize bootstrapping strategy



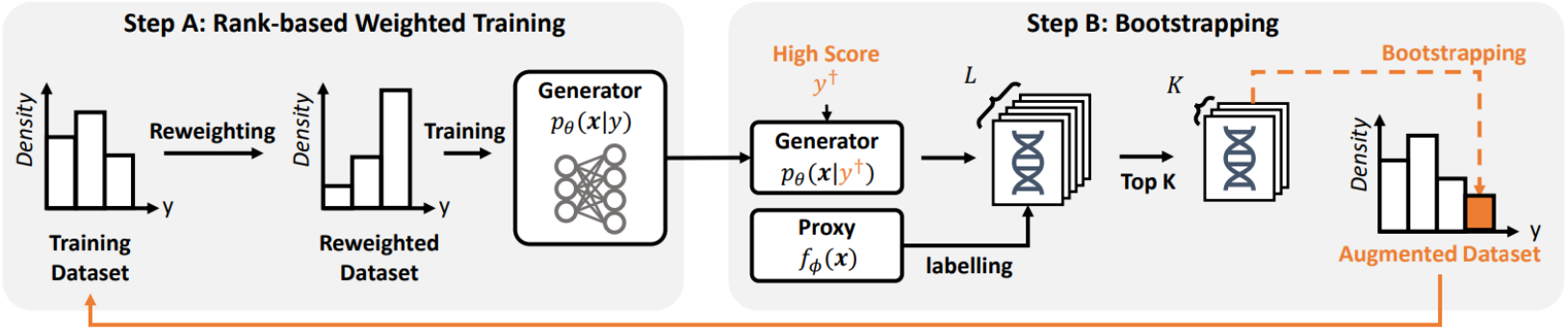


Figure 2.1: Illustration of the bootstrapped training process for learning score-conditioned generator.

2. BootGen

- **Algorithm Pseudocode**

Algorithm 1 Bootrapped Training of Score-conditioned generators

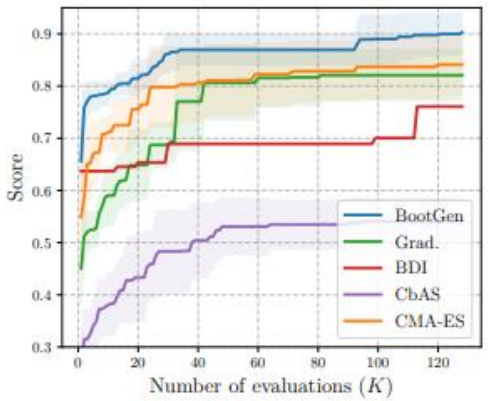
Input: Offline dataset $\mathcal{D} = \{\boldsymbol{x}_n, y_n\}_{n=1}^{N}$.
Update ϕ to minimize $\sum_{(\boldsymbol{x},y)\in\mathcal{D}}(f_\phi(\boldsymbol{x}) - y)^2$.
for $j = 1, \ldots, N_{\text{gen}}$ **do** ▷ Training multiple generaters
    Initialize $\mathcal{D}_{\text{tr}} \leftarrow \mathcal{D}$.
    for $i = 1, \ldots, I$ **do** ▷ Rank-based weighted training
        Update θ_n to maximize $\sum_{(\boldsymbol{x},y)\in\mathcal{D}_{\text{tr}}} w(y, \mathcal{D}_{\text{tr}}) \log p_{\theta_j}(\boldsymbol{x}|y)$.
        Sample $\boldsymbol{x}_\ell^* \sim p_{\theta_n}(\boldsymbol{x}|y^\dagger)$ for $\ell = 1, \ldots, L$. ▷ Bootstrapping
        Set $y_\ell^* \leftarrow f_\phi(x_\ell^*)$ for $\ell = 1, \ldots, L$.
        Set $\mathcal{D}_{\text{aug}}$ as top-K scoring samples in $\{\boldsymbol{x}_\ell^*, y_\ell^*\}_{\ell=1}^{L}$.
        Set $\mathcal{D}_{\text{tr}} \leftarrow \mathcal{D}_{\text{tr}} \cup \mathcal{D}_{\text{aug}}$.
    end for
end for
Output: trained score-conditioned generators $p_{\theta_1}(\boldsymbol{x}|y), \ldots, p_{\theta_N}(\boldsymbol{x}|y)$.

3. Experiment Results

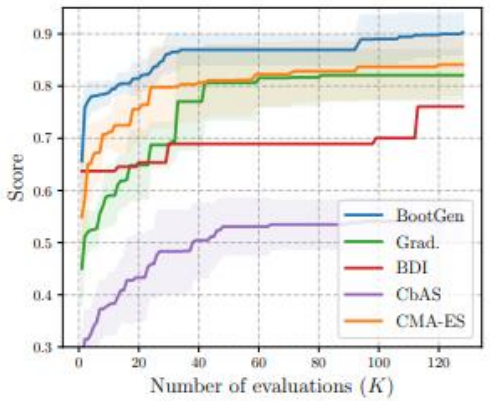
- **Outperform state-of-the-art baselines**

Table 4.1: Experimental results on 100th percentile scores. The mean and standard deviation are reported for 8 independent solution generations. $\mathcal{D}$(best) indicate the maximum score of the offline dataset. The best-scored value is marked in bold.

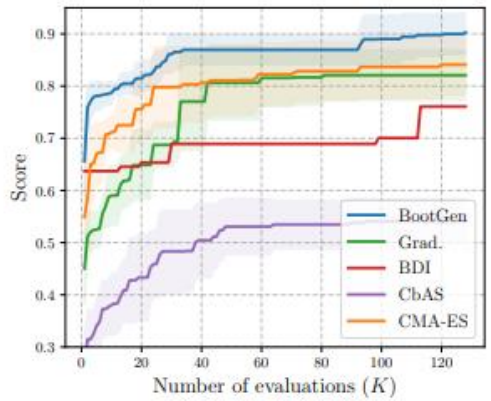
Method	RNA-A	RNA-B	RNA-C	TFBind8	GFP	UTR	Avg.
$\mathcal{D}$ (best)	0.120	0.122	0.125	0.439	0.789	0.593	0.365
REINFORCE [40]	0.462 ± 0.080	0.437 ± 0.033	0.463 ± 0.043	0.936 ± 0.041	**0.865** ± 0.003	0.685 ± 0.012	0.643
CMA-ES [21]	0.841 ± 0.058	0.822 ± 0.046	0.803 ± 0.039	0.904 ± 0.040	0.055 ± 0.003	0.737 ± 0.013	0.694
BO-qEI [45]	0.724 ± 0.055	0.729 ± 0.038	0.707 ± 0.034	0.798 ± 0.083	0.254 ± 0.352	0.684 ± 0.000	0.649
CbAS [10]	0.541 ± 0.042	0.647 ± 0.057	0.644 ± 0.071	0.913 ± 0.025	**0.865** ± 0.004	0.692 ± 0.008	0.717
Auto. CbAS [17]	0.524 ± 0.055	0.562 ± 0.031	0.495 ± 0.048	0.890 ± 0.050	**0.865** ± 0.003	0.693 ± 0.009	0.672
MIN [29]	0.376 ± 0.039	0.374 ± 0.041	0.404 ± 0.047	0.892 ± 0.060	**0.865** ± 0.001	0.691 ± 0.011	0.600
Grad [40]	0.821 ± 0.048	0.720 ± 0.047	0.688 ± 0.035	0.965 ± 0.030	0.862 ± 0.003	0.682 ± 0.013	0.792
COMs [39]	0.403 ± 0.062	0.393 ± 0.076	0.494 ± 0.098	0.945 ± 0.033	0.861 ± 0.009	0.699 ± 0.011	0.633
GFN-AL [26]	0.630 ± 0.054	0.677 ± 0.079	0.623 ± 0.045	0.956 ± 0.018	0.059 ± 0.006	0.695 ± 0.021	0.607
BDI [12]	0.700 ± 0.000	0.560 ± 0.000	0.632 ± 0.000	0.973 ± 0.000	0.864 ± 0.000	0.667 ± 0.000	0.733
BootGen	**0.902** ± 0.039	**0.931** ± 0.055	**0.831** ± 0.044	**0.979** ± 0.001	**0.865** ± 0.000	**0.865** ± 0.000	**0.895**



(a) RNA-Binding-A



(b) RNA-Binding-B



(c) RNA-Binding-C

3. Experiment Results

- **Not only achieve highest performance, but also have high novelty and diversity**

$$\text{Diversity}(\mathcal{D}) := \frac{1}{|\mathcal{D}|(|\mathcal{D}|-1)} \sum_{\boldsymbol{x}\in\mathcal{D}} \sum_{\boldsymbol{s}\in\mathcal{D}\setminus\{\boldsymbol{x}\}} d(\boldsymbol{x},\boldsymbol{s})\,.$$

$$\text{Novelty}(\mathcal{D},\mathcal{D}_{\text{offline}}) = \frac{1}{|\mathcal{D}|} \sum_{\boldsymbol{x}\in\mathcal{D}} \min_{\boldsymbol{s}\in\mathcal{D}_{\text{offline}}} d(\boldsymbol{x},\boldsymbol{s})\,.$$

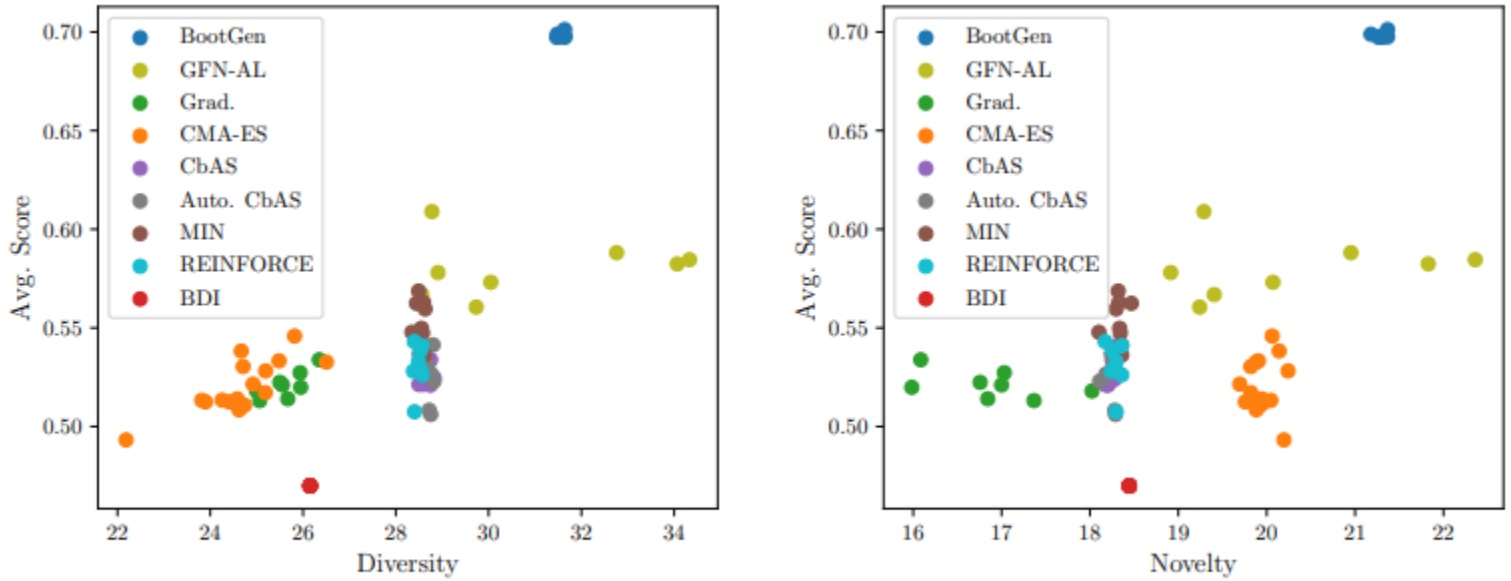


Figure 4.2: Multi-objectivity comparison of diversity and novelty on the average score for the UTR task. Each datapoint for 8 independent runs is depicted.

References

[1] Design-Bench: Benchmarks for Data-Driven Offline Model-Based Optimization (https://arxiv.org/abs/2202.08450)
[2] What are Diffusion Models? – Lil'Log (https://lilianweng.github.io/posts/2021-07-11-diffusion-models)
[3] Stanford CS231N Lecture 13. Generative Models (http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture13.pdf)